

Objetivo de la clase

Comprender los principales retos que aparecen cuando una base de datos debe manejar grandes volúmenes de información, múltiples usuarios y distintos servidores.

Durante la clase analizaremos:

- escalabilidad vertical y horizontal;
- bases de datos distribuidas;
- replicación y particionamiento;
- consistencia y concurrencia;
- teorema CAP;
- procesamiento paralelo.

Punto de partida

Supongamos una plataforma que almacena:

- perfiles de usuarios;
- publicaciones;
- imágenes;
- comentarios;
- registros de actividad.

Al principio, un solo servidor puede ser suficiente.

Pregunta

¿Qué sucede cuando millones de usuarios acceden al sistema y generan información de manera simultánea?

Cuando el sistema comienza a crecer

El crecimiento puede presentarse de diferentes formas:

- más usuarios;
- mayor volumen de información;
- más consultas por segundo;
- más operaciones simultáneas;
- mayor necesidad de almacenamiento;
- mayores requerimientos de procesamiento.

Este crecimiento obliga a pensar en cómo aumentar la capacidad del sistema.

¿Qué es la escalabilidad?

La escalabilidad es la capacidad de un sistema para mantener un funcionamiento adecuado cuando aumenta la demanda.

La demanda puede aumentar por:

- mayor cantidad de usuarios;
- crecimiento del volumen de datos;
- aumento del número de consultas;
- mayor cantidad de operaciones simultáneas;
- necesidad de procesar información con mayor rapidez.

Escalabilidad vertical y horizontal

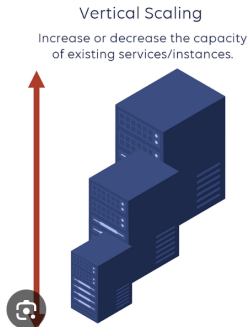
Escalabilidad vertical

Aumentar los recursos de un mismo servidor.

- Más memoria RAM.
- Más CPU.
- Mayor almacenamiento.

Escalabilidad horizontal

Agregar más servidores o nodos al sistema.



Horizontal Scaling

Add more resources like virtual machines to your system to spread out the workload across them.



Comparación de escalabilidad

Problema	Vertical	Horizontal
Poca memoria	Agregar RAM	Agregar nodos
Muchas consultas	Mejorar CPU	Distribuir consultas
Muchos datos	Aumentar disco	Distribuir información
Alta demanda	Mejorar servidor	Repartir carga

Idea principal

La escalabilidad horizontal permite aumentar la capacidad del sistema mediante la incorporación de nuevos nodos.

Una base de datos distribuida almacena y gestiona información utilizando varios nodos o servidores conectados mediante una red.

Los datos pueden encontrarse:

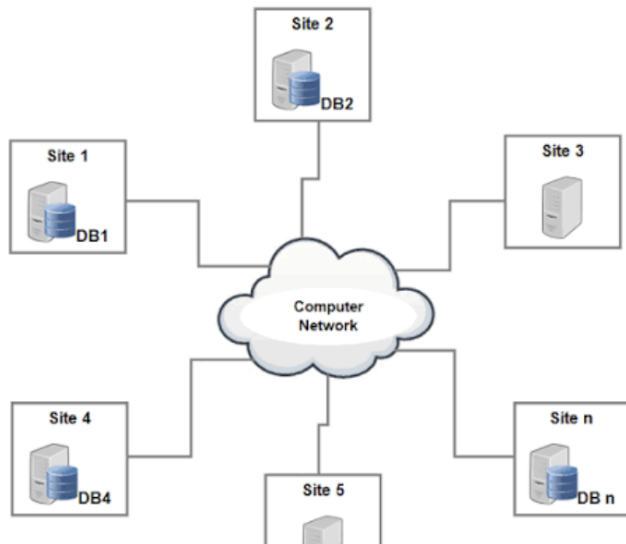
- distribuidos entre diferentes servidores;
- replicados en varios nodos;
- almacenados en diferentes ubicaciones físicas;
- administrados como parte de un mismo sistema.

¿Cómo se ve una base de datos distribuida?

En un sistema distribuido, varios nodos participan en el almacenamiento y procesamiento de la información.

Cada nodo puede:

- almacenar datos;
- procesar consultas;
- comunicarse con otros nodos;
- encontrarse en diferentes ubicaciones.



¿Qué es un nodo?

En un sistema distribuido, cada servidor participante puede considerarse un **nodo**.

Un nodo puede:

- almacenar una parte de la información;
- mantener una copia de los datos;
- recibir consultas;
- realizar operaciones;
- comunicarse con otros nodos.

Idea importante

Los nodos trabajan de manera coordinada para formar un único sistema lógico.

La replicación consiste en mantener copias de la misma información en diferentes nodos.

Objetivo

Aumentar la disponibilidad de los datos y reducir el riesgo de pérdida de información ante fallos.

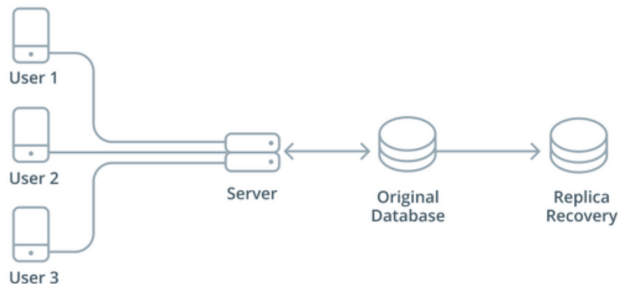
La replicación también puede ayudar a distribuir algunas operaciones de lectura.

Ejemplo de replicación

La información puede copiarse a uno o varios nodos adicionales.

Esto permite:

- mantener copias disponibles;
- recuperarse ante fallos;
- aumentar la disponibilidad.



Ventajas y retos de la replicación

Ventajas

- Mayor disponibilidad.
- Tolerancia a fallos.
- Copias de los datos.
- Distribución de lecturas.

Retos

- Sincronización.
- Mayor uso de almacenamiento.
- Retrasos entre réplicas.
- Posibles inconsistencias temporales.

Particionamiento de datos

El particionamiento consiste en dividir la información en diferentes partes.

Por ejemplo:

Nodo 1	Nodo 2	Nodo 3
Usuarios 1–1000	Usuarios 1001–2000	Usuarios 2001–3000

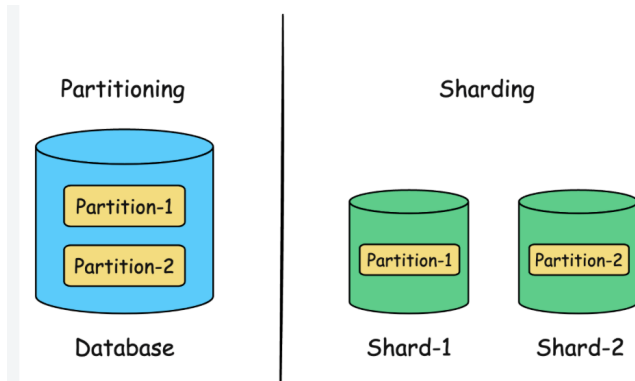
Cada nodo puede almacenar sólo una parte del conjunto total de datos.

Particionamiento y sharding

El particionamiento divide la información en partes.

Cuando estas partes se distribuyen entre distintos nodos o servidores, se utiliza comúnmente el término:

Sharding



Replicación vs. particionamiento

Replicación

- Se crean copias.
- Los mismos datos pueden aparecer en varios nodos.
- Mejora la disponibilidad.

Particionamiento

- Se dividen los datos.
- Cada nodo almacena una parte.
- Permite distribuir grandes volúmenes.

En un sistema distribuido pueden existir varias copias de la información.

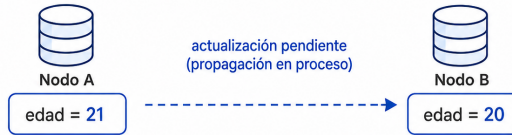
Cuando un dato cambia en un nodo, la actualización debe propagarse hacia otros nodos.

Pregunta

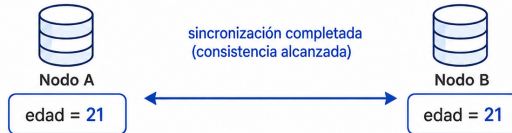
¿Qué ocurre si durante algunos segundos dos nodos contienen valores diferentes?

Consistencia

1 Después de una actualización



2 Después de la sincronización



Las réplicas deben converger hacia una misma versión de los datos.

Consistencia entre réplicas

Supongamos que inicialmente tenemos:

Nodo A: edad = 20

Nodo B: edad = 20

Después de una actualización:

Nodo A: edad = 21

Nodo B: edad = 20

Después de la sincronización:

Nodo A: edad = 21

Nodo B: edad = 21

La concurrencia ocurre cuando múltiples usuarios o procesos intentan acceder o modificar información al mismo tiempo.

Por ejemplo:

Producto disponible: 1

Usuario A

Usuario B

Comprar

Comprar

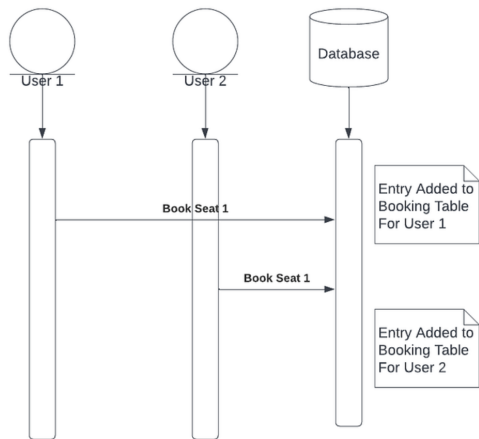
Ambos usuarios podrían intentar comprar el mismo producto simultáneamente.

Ejemplo de concurrencia

Dos usuarios intentan reservar el mismo asiento.

Si las operaciones no se coordinan correctamente, podría ocurrir:

- una doble reserva;
- una actualización perdida;
- un conflicto entre operaciones.



Si la concurrencia no se controla adecuadamente, pueden aparecer problemas como:

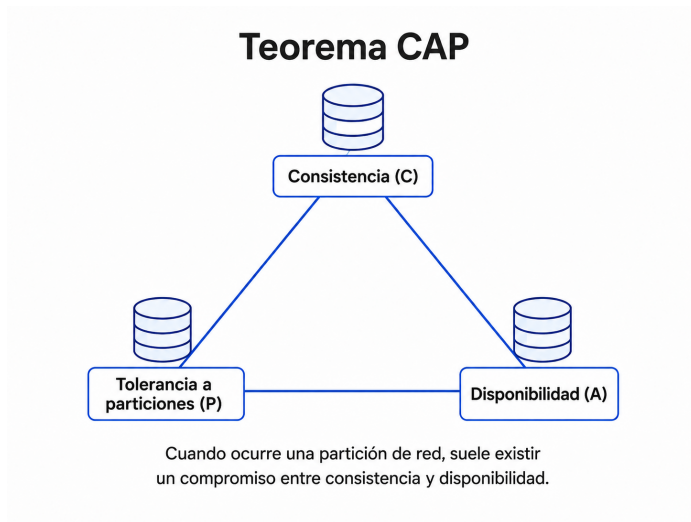
- actualizaciones perdidas;
- datos inconsistentes;
- operaciones duplicadas;
- lecturas de información desactualizada;
- conflictos entre operaciones simultáneas.

El teorema CAP describe tres propiedades importantes en sistemas distribuidos.

C – Consistency Los nodos ofrecen una visión coherente de la información.

A – Availability Cada solicitud recibe una respuesta del sistema.

P – Partition Tolerance El sistema continúa operando aunque exista una interrupción de comunicación entre algunos nodos.



¿Qué es una partición de red?

Una partición ocurre cuando algunos nodos dejan de comunicarse entre sí.

Nodo A

×

Nodo B

Los nodos pueden continuar funcionando, pero ya no pueden intercambiar información correctamente.

El compromiso del teorema CAP

Cuando ocurre una partición de red, el sistema debe decidir cómo responder.

Priorizar consistencia

El sistema puede evitar responder si no puede garantizar que la información está actualizada.

Priorizar disponibilidad

El sistema puede continuar respondiendo aunque algunos nodos tengan temporalmente información diferente.

Importante

CAP no debe interpretarse simplemente como “elegir dos de tres” en cualquier circunstancia.

Ejemplo conceptual de CAP

Sistema bancario

Puede ser más importante evitar mostrar información financiera incorrecta.

Prioridad posible: consistencia.

Red social

Puede ser aceptable que una publicación tarde unos segundos en aparecer en todos los nodos.

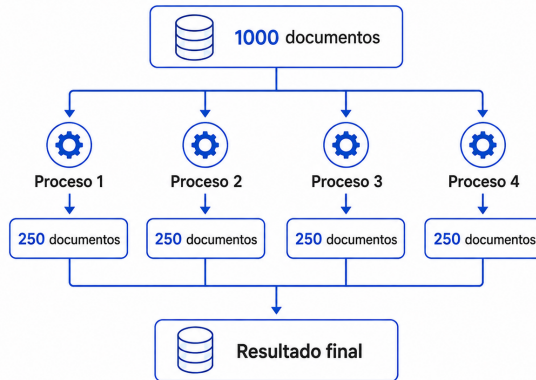
Prioridad posible: disponibilidad.

El procesamiento paralelo consiste en dividir una tarea en varias partes que pueden ejecutarse simultáneamente.

Puede utilizarse para:

- procesar grandes volúmenes de información;
- reducir tiempos de ejecución;
- distribuir tareas entre varios procesadores;
- aprovechar varios nodos de un sistema;
- ejecutar diferentes operaciones al mismo tiempo.

Procesamiento paralelo



Una tarea grande se divide en subtarefas que se ejecutan simultáneamente.

Procesamiento paralelo: ejemplo

Supongamos que debemos procesar:

1000 documentos

Podemos dividir la tarea en cuatro procesos:

Proceso 1	Proceso 2	Proceso 3	Proceso 4
250	250	250	250

Los cuatro procesos pueden ejecutarse simultáneamente y posteriormente combinar sus resultados.

Crecimiento de datos y usuarios



Escalabilidad



Bases de datos distribuidas



Replicación y particionamiento



Consistencia, concurrencia y CAP

- Gilbert, S., & Lynch, N. (2002). *Brewer's Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services*. ACM SIGACT News, 33(2), 51–59.
- DeCandia, G., Hastorun, D., Jampani, M., Kakulapati, G., Lakshman, A., Pilchin, A., Sivasubramanian, S., Vosshall, P., & Vogels, W. (2007). *Dynamo: Amazon's Highly Available Key-value Store*. Proceedings of the 21st ACM Symposium on Operating Systems Principles (SOSP).
- Dean, J., & Ghemawat, S. (2004). *MapReduce: Simplified Data Processing on Large Clusters*. Proceedings of the 6th USENIX Symposium on Operating Systems Design and Implementation (OSDI), 137–150.